

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



INTERNSHIP REPORT ON

“QUALITY ASSURANCE AND TEST AUTOMATION FOR THE BROADCAST MODULE IN AN AI CHATBOT PLATFORM”

Submitted in partial fulfillment for the award of the degree of

Bachelor of Engineering

In

Artificial Intelligence and Machine Learning

Submitted by:

Harshitha S - 1DS21AI021

Undertaken at



**Engati Technologies Private Limited,
Bangalore, Karnataka**

**Department of Artificial Intelligence and Machine Learning,
Dayananda Sagar College of Engineering, Bengaluru**

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India



CERTIFICATE

This is to certify that the Internship titled “**Quality Assurance and Test Automation for the Broadcast Module in an AI Chatbot Platform**” carried out by **Ms HARSHITHA S [1DS21AI021]**, a bonafide student of **Dayananda Sagar College of Engineering**, in partial fulfillment for the award of **Bachelor of Engineering in Artificial Intelligence and Machine Learning** during the academic year **2024–2025**. It is certified that all corrections/suggestions indicated have been incorporated in the report. The internship report has been approved as it satisfies the academic requirements in respect of the course Internship (21INT82).

Signature of the
Reporting Manager
Krubha Velamurugan,
Manager, QA,
Engati Technologies

Signature of the
Internship Coordinator
Dr. Reshma S,
Assistant Professor,
Dept. of AI & ML, DSCE

Signature of the
Head of the Department
Dr. Vindhya P Malagi,
Professor & Head,
Dept. of AIML, DSCE

Name of the Examiners

Signature with date

1.

.....

2.

.....

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DECLARATION

I, **Harshitha S**, a final year student of **Artificial Intelligence and Machine Learning, Dayananda Sagar College of Engineering, Bengaluru**, hereby declare that this internship report is based on the work undertaken during my ongoing internship in the role of **Intern, Quality Analyst** at **Engati Technologies Private Limited, Bangalore, Karnataka**.

This report is submitted in partial fulfillment of the requirements for the award of the Bachelor's Degree in Artificial Intelligence and Machine Learning during the academic year 2024–2025.

Date: 12/04/2025

Place: Bengaluru

OFFER LETTER



CIN: U72900KA2020FTC134174

Date : 10th June,
2024 Name :
Harshitha S

Dear **Harshitha S**,

Engati Technologies Pvt Ltd ("Engati" or "Company") is pleased to offer you an **"QA Intern"** position with the Company for a period starting from **17th Jul, 2024 to 17th Jan, 2025**. During your internship, you will be working on the assignment at our Bangalore office and paid a stipend of **INR 25,000/-** per month subjected to taxes.

This is a probationary offer, which does not imply any commitment by Engati for regular employment. At the end of the Probation Period, successful candidates will be eligible for full time employment, to be offered based on performance and at the sole discretion of Engati.

In case you decide to separate from Engati before completing your probation period, you would be required to give one (1) month of notice to the Company. The Company may, at its option and discretion, waive the prescribed notice period or a part thereof in case of a voluntary separation. Company reserves the right to terminate this agreement with immediate effect if your performance is found unsatisfactory. Further, Probation completion letter (on request) will be provided only on completion of the Probation Period.

Information pertaining to Engati including and not limited to commercial, business, financial data, sales, pricing, technical, operational, administrative, marketing or other information or data (including, intellectual property, know-how, customer and supplier details are confidential and upon reporting, you will be required to sign a non-disclosure agreement. You are also expected to adhere to all applicable rules, regulations and business conduct guidelines of Engati and act in accordance with the values and principles of the Company. Please note that the stipend is payable during the probation period is subject to your 21 full working days at Engati.

To confirm your acceptance of this assignment, you are required to:

- Respond via email to swapna@quinbay.com to communicate acceptance of the offer and to confirm your joining date.
- Any change of joining date must be sent to swapna@quinbay.com at least 7 days prior to your original joining date. The new joining date must be no later than 5 days from the date of joining listed above.
- Report for on-boarding at **10 a.m. at 1029, 24th Main Rd, 1st Sector, HSR Layout, Bengaluru, Karnataka 560102**

ENGATI TECHNOLOGIES PRIVATE LIMITED
1029, 24th Main Rd, 1st Sector, HSR Layout, Bengaluru,

COMPLETION CERTIFICATE



CIN: U72900KA2020FTC134174

Date : 10th Jan, 2025

TO WHOM IT MAY CONCERN

This is to certify that **Harshitha S** has done her internship as **QA Intern** at **Engati Technologies Pvt Ltd, Bangalore** starting **17-Jul-24 to 20-Dec-24**. During the internship period she had been punctual, hardworking and inquisitive.

We wish the very best in all future endeavors and will cherish the association with Engati.

Sincerely,

Swapna Sunandita Samal
Senior HR Manager

Harshitha S

ACKNOWLEDGEMENT

This internship is the result of continuous guidance, support, and encouragement from several individuals, to whom I am deeply grateful. I take this opportunity to express my sincere appreciation to everyone who contributed to the successful completion of this internship.

I would like to express my heartfelt thanks to **Dr. B G Prasad**, Principal, Dayananda Sagar College of Engineering, for providing the necessary facilities and support to undertake this internship.

My sincere gratitude goes to **Dr. Vindhya P Malagi**, Head of the Department – Artificial Intelligence & Machine Learning, for providing me the opportunity to pursue this internship and for her valuable guidance and encouragement throughout.

I am thankful to **Dr. Reshma S**, Internship Coordinator and Assistant Professor – AI & ML, for her continuous support, timely suggestions, and motivation during the internship period.

I would also like to extend my thanks to **Prof. Kavya N**, internal guide, for her consistent guidance, timely feedback, and unwavering support throughout the internship. Her mentorship helped me stay aligned with both academic and industry expectations.

I would like to thank **Kruba Velamurugan**, Manager, Engati Technologies, for the valuable industry insights and mentorship provided during my role as an intern in Marketing Analytics.

My thanks also go to all the faculty members and non-teaching staff of the Department of AI & ML for their cooperation and assistance during the internship period.

HARSHITHA S [1DS21AI021]

TABLE OF CONTENTS

Sl no	Description	Page No
1	Introduction	1
2	About the Company	2
3	Scope of the Work	3
4	Learning Objectives	5
5	Challenges & Solutions	7
6	Achievements & Contributions	8
7	Reflection and Analysis	10
8	Recommendations	11
9	Conclusions	12
10	References	

CHAPTER 1

INTRODUCTION

Testing played a vital role in ensuring the reliability, accuracy, and performance of the Broadcast module within Engati's AI chatbot platform. The primary objective of the project was to validate the seamless delivery of bulk messages across various communication channels while ensuring the system could handle a diverse range of scenarios without failure. To achieve this, a comprehensive testing strategy was employed, incorporating manual, automated, and performance testing techniques. This multifaceted approach contributed significantly to the robustness of the module and enhanced the overall user experience.

Manual testing was used extensively for exploratory and scenario-based testing to validate core functionalities such as message creation, scheduling, and delivery workflows. This helped identify UI inconsistencies, unexpected behaviors, and edge cases that might otherwise go unnoticed. Regression testing was performed regularly to ensure that new code changes did not impact existing features, preserving functionality across software versions. Sanity testing was conducted after each build or deployment to perform quick checks on critical components, serving as an initial gatekeeper for further testing.

On the automation front, UI automation was implemented using tools such as Selenium and Cypress, enabling efficient execution of repetitive test cases and ensuring cross-browser compatibility. In parallel, API automation was used to validate backend endpoints responsible for scheduling, targeting, and message delivery logic, ensuring both functionality and data integrity. Altogether, this structured testing approach played a crucial role in maintaining the stability and performance of the Broadcast module, ensuring it met user expectations and business requirements.

As part of my academic curriculum and career development in the field of software engineering, I undertook a **Software Development Engineer in Test (SDET)** internship at **Engati Technologies**, a leading conversational AI platform based in Bengaluru. This internship provided me with the opportunity to apply my theoretical knowledge in a practical, fast-paced, and collaborative environment. Over the course of my internship, I worked with the Quality Engineering team to understand and contribute to the processes involved in testing AI-powered chatbot platforms and web applications.

The internship focused on key areas such as manual testing, automation scripting, bug tracking, and test case execution within an agile software development lifecycle. My responsibilities included writing and executing test cases, developing automated test scripts using Nightwatch.js and Cucumber, performing cross-browser testing with BrowserStack, and logging defects in JIRA. I was also involved in daily stand-ups, sprint planning meetings, and retrospectives, which gave me valuable insight into the workflow of a professional software development team.

Through this experience, I was exposed to real-world challenges, tight deadlines, and the importance of delivering high-quality software. The internship helped me build a strong foundation in test automation frameworks, agile methodologies, version control systems, and the end-to-end software quality assurance process. It also sharpened my problem-solving abilities and taught me how to think critically from a tester's perspective.

Overall, my internship at Engati Technologies was an enriching experience that bridged the gap between academic learning and industry expectations. It not only enhanced my technical competencies but also gave me clarity and confidence in pursuing a career in software quality engineering.

CHAPTER 2

ABOUT THE COMPANY



fig. 2.1 Company Logo

Engati Technologies Private Limited is a leading provider of AI-native customer experience (CX) solutions, specializing in conversational automation to help businesses engage with customers across various digital platforms. Founded in 2016, Engati has developed a robust platform that enables the creation and deployment of chatbots and live chat systems, facilitating omnichannel engagement and multilingual conversations. The company's solutions are widely used in industries such as e-commerce, logistics, education technology, travel, insurance, and real estate.

Engati's platform supports integration across multiple channels, including WhatsApp, Facebook, Instagram, and websites, allowing businesses to automate customer interactions effectively. With a focus on low-code bot building and advanced natural language processing (NLP) capabilities, Engati empowers organizations to create intelligent conversational agents that enhance customer engagement and streamline support services.

Headquartered in San Jose, United States, with a significant presence in Bengaluru, India, Engati continues to expand its global footprint, serving a diverse clientele seeking innovative AI-driven customer engagement solutions.

Engati's platform offers a comprehensive suite of features, including WhatsApp Chatbot, Shopify Chatbot, AI Chatbot, Instagram Chatbot, Live Chat, and eSenseGPT. These tools enable omnichannel engagement, multilingual conversations, chatbot training, and seamless integrations, catering to industries such as e-commerce, logistics, edtech, healthcare, travel, insurance, and real estate.

As of 2022, Engati Technologies Private Limited reported a revenue of ₹21.3 crore and employed 127 individuals, reflecting its significant growth and impact in the AI-driven customer engagement space.

Engati Technologies is a leading global platform specializing in conversational automation solutions powered by artificial intelligence. Headquartered in Bengaluru, India, the company was founded with the vision of enabling businesses to build and deploy intelligent chatbots and live chat systems without writing a single line of code. Since its inception in 2020, Engati has grown into a well-recognized player in the AI-driven customer experience space.

Engati's flagship product is a no-code platform that allows businesses to create conversational bots capable of handling a wide range of customer service and engagement tasks. These bots can be deployed across multiple channels, including WhatsApp, Facebook Messenger, Instagram, Telegram, Viber, Microsoft Teams, websites, and mobile apps. This omnichannel capability makes Engati an attractive solution for businesses looking to provide seamless customer support and engagement across all digital touchpoints.

In 2024, Engati was awarded **Meta's Tech Partner of the Year** in recognition of its innovations and leadership in developing powerful WhatsApp-based solutions. This award highlighted the company's commitment to delivering high-quality, scalable, and secure communication tools for businesses of all sizes. Its WhatsApp automation framework is especially popular among businesses aiming to reach and support customers efficiently through one of the world's most widely used messaging platforms.

One of the most notable innovations from Engati is **eSenseGPT**, an enterprise-level conversational automation tool built on top of large language models (LLMs) such as GPT-4 and Google's Bard AI. This tool enables more natural and intelligent interactions by allowing bots to understand user intent, context, and even nuanced language. eSenseGPT helps enterprises automate repetitive tasks, improve lead conversion, and reduce response time without sacrificing personalization.

Engati's platform is used in a wide range of industries including e-commerce, logistics, travel, insurance, banking, education, and real estate. Businesses in these sectors benefit from Engati's ability to integrate AI-powered bots into their existing workflows to improve operational efficiency and customer satisfaction. With features like multilingual support, quick integrations, analytics dashboards, and proactive messaging, Engati helps organizations enhance customer engagement while reducing support costs.

In addition to its chatbot platform, Engati also offers live chat and marketing automation tools that help customer-facing teams connect with users in real time. This unified solution allows for a smooth handoff between bots and human agents, ensuring that complex queries are handled effectively.

Engati's engineering and product development teams are constantly innovating to improve platform performance, security, and usability. For instance, the platform offers enterprise-grade compliance with data protection standards, making it a reliable choice for regulated industries. It also integrates seamlessly with CRM tools, payment gateways, and cloud platforms, making deployment and scaling simple.

Overall, Engati Technologies stands out as a pioneer in the conversational AI space, helping businesses transform how they interact with their customers. Its focus on ease of use, intelligent automation, and channel flexibility positions it as a key enabler of digital transformation in customer experience.

CHAPTER 3

SCOPE OF THE WORK

During my six-month internship as a **Software Development Engineer in Test (SDET) Intern** at **Engati Technologies Private Limited**, I was actively involved in a wide range of testing activities aimed at ensuring the quality, reliability, and performance of the **Broadcast Module**, a key component of Engati's AI-powered chatbot platform. My responsibilities spanned across multiple phases of the software testing lifecycle, involving both manual and automation testing, along with performance evaluations and API validations. The following points elaborate on the specific areas and activities that formed the core scope of my work:

1. Understanding the Product Architecture:

- Gained a comprehensive understanding of **Engati's conversational AI platform**, with special focus on the **Broadcast Module** used to deliver bulk messages across platforms like WhatsApp, Facebook Messenger, web chat, and more.
- Studied the functional and technical specifications of the module to understand business objectives, message delivery workflows, and system interactions.

2. Test Planning and Strategy Development:

- Participated in requirement analysis meetings and sprint planning discussions.
- Contributed to the creation of **test plans, test cases, and testing strategies** for both new features and enhancements related to the Broadcast system.
- Prioritized test cases based on critical functionalities and risk factors to ensure optimal test coverage during each sprint cycle.

3. Regression and Sanity Testing

- Conducted **regression testing** regularly to verify that new changes or fixes did not break existing functionalities.
- Led and executed **sanity tests** on major builds to ensure the system was stable enough for further QA activities.

4. Test Automation

- Developed and maintained **automated test scripts** using:
- **Selenium** for web-based UI automation testing.

5. API Testing

- Used Postman and Cypress API automation to validate backend APIs responsible for message handling, scheduling, analytics, and delivery logs.
- Ensured data consistency between UI and backend responses.
- Verified authentication, response times, error handling, and edge cases in API responses.

5. Performance Testing

- High message volume
- Long-running schedules
- Simultaneous multi-channel delivery

This internship provided me with hands-on experience in a real-world agile software development environment and enhanced my skills in functional, automation, API, and performance testing. By working on the Broadcast module, I gained practical insights into how scalable communication features are built, tested, and delivered as part of a modern AI-powered platform. The diversity of tasks and collaborative exposure during this internship significantly contributed to my growth as a test engineer and strengthened my ability to contribute effectively in future SDET role.

During my six-month internship as a Software Development Engineer in Test (SDET) at Engati Technologies, my primary focus was on the **Broadcast module**, a key component of Engati's AI chatbot platform. This module enables businesses to send bulk messages to users across different communication channels like WhatsApp, Facebook Messenger, and web chat, often to share updates, offers, or alerts. Given its business-critical nature, the Broadcast module required rigorous validation to ensure accuracy, speed, and fault-tolerance. My role involved designing and executing test cases that evaluated various aspects of this module—from message creation to scheduling, targeting, and successful delivery across platforms.

My responsibilities included performing **manual testing**, **regression testing**, **sanity checks**, **UI automation**, **API testing**, and **performance benchmarking**. I explored real-world user scenarios and created test data to simulate edge cases, ensuring the system behaved reliably even under unusual conditions. Using tools like **Selenium** and **Cypress**, I developed automation scripts to speed up repetitive testing workflows and validate browser compatibility. On the backend, I used **Postman** to verify API responses and assert the accurate high user traffic and large-scale broadcasting conditions.

Beyond technical execution, my scope of work extended to documenting bugs, tracking issues using **JIRA**, collaborating with developers to resolve defects, and participating in daily stand-ups and sprint reviews as part of the **Agile development process**. I also worked on optimizing existing test suites and improving test coverage metrics. This end-to-end involvement not only helped me gain exposure to the full QA lifecycle but also allowed me to contribute meaningfully to the product's stability, reliability, and customer satisfaction. Overall, my work played a vital role in helping the Broadcast module meet quality benchmarks before being released to production.

1. Automation VS Manual testing-

Manual Testing is the process of executing test cases by a human tester without using any automated tools. It involves checking the functionality of an application by manually interacting with the software, following test scenarios, and comparing actual outcomes with expected results. Manual testing is often used for exploratory, usability, and ad-hoc testing where human intuition, visual cues, and real-time decision-making are essential. It is especially useful during the early stages of development or for short-term projects where automation may not be cost-effective.

Automation Testing, on the other hand, uses tools, scripts, and frameworks to run tests automatically. It is ideal for regression testing, performance testing, and large-scale projects where repetitive test cases need to be executed frequently and consistently. Automation increases test coverage, improves accuracy, and significantly reduces execution time, especially when tests are run across multiple platforms or devices. Popular tools like Selenium, Nightwatch.js, and Cypress help QA teams write and execute test scripts with minimal human intervention.

A major difference lies in **efficiency and scalability**. While manual testing requires more time and effort for repetitive tasks, automation enables faster execution, making it suitable for continuous integration and continuous delivery (CI/CD) environments. However, automation also requires an initial investment of time and resources to set up scripts, maintain test data, and update code with application changes. In contrast, manual testing has a lower setup cost and can be more flexible when requirements are not clearly defined or are frequently changing.

In summary, both manual and automation testing play crucial roles in a well-rounded QA strategy. Manual testing is best suited for test cases that require human judgment and flexibility, while automation testing excels in handling repetitive, stable, and time-consuming tasks. The choice between them often depends on the project size, timelines, budget, and the specific goals of the testing phase. In agile environments, a balanced combination of both methods is typically adopted to ensure speed, quality, and reliability.

2. Test Case Design Techniques -

During my internship as an SDET at Engati Technologies, I learned and applied several **test case design techniques** to ensure comprehensive and effective test coverage. These techniques helped me design test cases that could detect defects early, reduce redundant testing, and validate both functional and non-functional aspects of the application.

One of the fundamental techniques I used was **Equivalence Partitioning**, which involves dividing input data into valid and invalid partitions and selecting representative values from each partition. This technique minimized the number of test cases while maximizing coverage, especially when testing input fields or APIs with defined data constraints.

I also applied **Boundary Value Analysis (BVA)**, which focuses on testing the boundaries of input ranges. Since bugs often occur at the edges of input domains, this method allowed me to identify edge-case defects by testing values at, just below, and just above the boundaries.

Another important technique I practiced was **Decision Table Testing**, especially for features that required testing multiple conditions and their respective outcomes. By representing business rules in tabular form, this technique helped me design clear and systematic test cases for complex logic and combinations.

Additionally, I used **State Transition Testing** for verifying how the application behaved in different states and transitions—this was particularly useful for testing the chatbot’s conversation flows, where user input determines the next response or transition. These test design strategies enabled me to approach testing more strategically, ensuring that both common and rare scenarios were covered effectively.

3. API Testing

During my internship at Engati Technologies, I was introduced to **API Testing**, which plays a crucial role in ensuring that backend services and system integrations function as expected. Since Engati's platform relies on seamless communication between frontend interfaces and backend services, API testing became an important part of validating the overall application behavior.

I primarily used **Postman**, a widely adopted tool for testing RESTful APIs. With Postman, I was able to send various HTTP requests (GET, POST, PUT, DELETE) to the endpoints, inspect responses, validate response codes, headers, and body data, and ensure that the APIs returned the expected output. This was especially helpful in early-stage testing before the UI was fully developed or integrated.

One of the key aspects of API testing I focused on was **validating response accuracy and performance**. I wrote test cases to verify not just the correctness of the data returned, but also the time it took to receive a response. I also checked how the APIs handled edge cases such as missing parameters, invalid inputs, and authorization failures. By testing these scenarios, I ensured the robustness and security of API endpoints.

Furthermore, I practiced creating **automated API test collections** in Postman using its built-in scripting capabilities and executed them using **Newman**, Postman's command-line companion. This allowed me to automate API tests as part of a CI pipeline and verify that new code changes did not break backend functionality. Through this experience, I gained valuable insight into how APIs support scalable applications and how thorough testing can prevent critical issues in production.

4. Understanding SDLC vs STLC

During my internship at Engati Technologies, I gained a deeper understanding of the software development lifecycle (SDLC) and the software testing lifecycle (STLC), and how both work together to ensure the delivery of high-quality software. While both lifecycles are crucial to a successful software project, they serve different purposes and involve different roles and activities.

The **Software Development Life Cycle (SDLC)** is a structured process followed by development teams to design, develop, test, and deploy software applications. It includes stages such as requirement gathering, system design, coding, testing, deployment, and maintenance. SDLC ensures that the final product meets business requirements, is built on time, and is maintainable.

At Engati, I observed how the development team followed an Agile SDLC model, where each sprint delivered a small increment of the product with iterative planning, development, and review cycles.

On the other hand, the **Software Testing Life Cycle (STLC)** is a set of specific steps carried out by the QA team to ensure the quality of the software through rigorous validation and verification. STLC includes phases such as test planning, test case development, environment setup, test execution, defect reporting, and test closure. These activities are integrated into the SDLC, often starting in parallel with requirement analysis and continuing through deployment. During my internship, I was actively involved in STLC phases such as writing test cases, executing test scenarios, and logging defects in JIRA for the features under development.

A key takeaway from my experience was that while SDLC focuses on the broader picture of software delivery, STLC zooms in on ensuring that each component is tested thoroughly and meets the quality standards. Both lifecycles are interdependent—without SDLC, there is no product to test, and without STLC, there is no structured validation to ensure quality. Understanding how both these lifecycles interact gave me a more holistic view of how modern software projects are planned, built, and released in an agile environment.

5. Code Review and Git Workflow

As part of my role as an SDET intern at Engati Technologies, I actively participated in the version control and collaboration process through **Git** and understood the importance of a structured **Git workflow**. Git enabled efficient collaboration between developers and testers by managing changes to the codebase, keeping track of history, and minimizing merge conflicts through a well-defined branching strategy.

The team followed a **feature-branch workflow**, where every new feature, bug fix, or test script update was implemented in a separate branch derived from the main development branch (often named **develop** or **main**). I created branches for writing automation scripts, pushed changes regularly, and submitted **pull requests (PRs)** for review. This process ensured that all changes were isolated, traceable, and peer-reviewed before being merged into the main codebase.

Code reviews played a crucial role in maintaining code quality and encouraging shared ownership. Senior QA engineers and developers reviewed my pull requests to verify script quality, structure, efficiency, and adherence to project conventions.

I received valuable feedback during this process, such as optimizing selectors in UI tests, improving assertion logic, and enhancing the readability of test code. I also reviewed others' code, which helped me understand diverse testing styles and best practices.

In addition to version control, we used **GitHub** or similar platforms to manage repositories, issues, and project boards. This integration allowed seamless linking of test cases, defects, and user stories, contributing to better traceability and collaboration. Through this experience, I not only improved my technical skills in Git but also learned the value of communication, accountability, and continuous improvement in a collaborative development environment.

6. Bug Life Cycle

During my SDET internship at Engati Technologies, I was introduced to the **Bug Life Cycle**, a critical process in software testing that tracks the life of a defect from its identification to its closure. Understanding this cycle helped me follow a disciplined and systematic approach to reporting, tracking, and retesting bugs throughout the development process.

The cycle begins when a tester identifies a defect during test execution and logs it in a bug-tracking tool like **JIRA**. At this stage, the bug is marked as "**New**", and relevant details such as summary, description, steps to reproduce, severity, priority, screenshots, and test environment are added to ensure clarity. The QA lead or product manager then reviews the bug and assigns it to the appropriate developer, transitioning the status to "**Assigned**".

Once the developer starts working on the bug, its status changes to "**In Progress**". After fixing the issue, the developer marks it as "**Fixed**", and the bug is sent back to the QA team for verification. As a tester, I would then **retest** the application to confirm whether the issue was truly resolved. If the fix was successful and the application behaved as expected, I marked the bug as "**Closed**". If not, I would **reopen** the bug and send it back for further investigation.

Additionally, some bugs could be marked as "**Rejected**", "**Deferred**", or "**Duplicate**" based on specific conditions. For instance, if a bug was found to be invalid or not reproducible, it might be rejected. If it was planned to be fixed in a later release, it would be deferred. Understanding these transitions helped me collaborate more effectively with developers and ensure that no defect was overlooked. This structured approach to defect tracking greatly improved the quality and reliability of the product.

CHAPTER 4

LEARNING OBJECTIVES

My internship as a Software Development Engineer in Test (SDET) Intern at Engati Technologies Private Limited provided me with a valuable opportunity to bridge the gap between academic knowledge and industry practices. The core objective of this internship was to develop a deep understanding of the software development and testing lifecycle in a real-world, agile environment. Through active involvement in testing the Broadcast Module of Engati's AI chatbot platform, I was able to gain hands-on experience in both manual and automated testing, performance evaluation, and API validation. This experience significantly enhanced my technical skills, professional mindset, and ability to work collaboratively in a fast-paced software development team.

One of the primary learning objectives was to develop a solid foundation in **quality assurance processes**, including test planning, execution, reporting, and defect management. I learned how to write effective and comprehensive test cases that covered both common usage scenarios and edge cases. I also gained the ability to analyze system requirements, identify potential areas of risk, and design testing strategies accordingly. Regular participation in sprint planning and review meetings enabled me to understand how testing aligns with broader development objectives and how early-stage planning contributes to overall product quality.

Another significant area of learning was the application of **automation tools and frameworks**. I became proficient in using **Selenium** and **Cypress** to automate web-based testing, which improved my understanding of how automated scripts are developed, maintained, and integrated into CI/CD pipelines. This helped me appreciate the value of automation in improving test coverage, saving time, and enabling continuous integration. Writing reusable and modular automation scripts taught me how to think systematically and maintain code quality, even within testing environments. I also learned how to debug automation failures and collaborate with developers to resolve root causes efficiently.

The internship also helped me understand the importance and application of **API testing**, particularly in the context of a microservices-based architecture. Using tools such as **Postman** and **Cypress for API automation**, I learned to validate RESTful APIs, examine request/response cycles, handle various status codes, and test backend functionalities independent of the UI. I became confident in verifying API contracts, authorization

mechanisms, and data flow between services. This knowledge helped me appreciate how front-end and back-end components interact in modern software systems, and how API testing plays a crucial role in ensuring seamless user experiences.

Performance testing was another key learning area during my internship. I explored the use of tools like **JMeter** and browser-based utilities to simulate load on the Broadcast Module, evaluate response times, and identify performance bottlenecks. I gained practical insights into how applications are expected to behave under heavy load and stress conditions, especially in modules like Broadcast that are responsible for bulk operations. This helped me understand system scalability and the significance of performance optimization in production-grade systems.

In addition to technical growth, this internship also helped me develop essential **soft skills and work ethics**. I learned the value of clear communication, documentation, and time management. Attending daily stand-ups and collaborative meetings allowed me to express my updates and blockers clearly, receive feedback, and contribute meaningfully to the team's progress. I also became familiar with **Agile methodologies**, sprint cycles, and tools like **JIRA** for project and bug tracking, giving me a clear understanding of how structured yet flexible project management practices work in professional settings.

Furthermore, I gained valuable exposure to **real-world challenges** that quality engineers face in dynamic environments, such as changing requirements, quick release cycles, and coordinating across cross-functional teams. Solving bugs under tight deadlines, adapting to toolchain changes, and debugging unfamiliar systems enhanced my adaptability, analytical thinking, and resilience under pressure.

Ultimately, this internship enabled me to evolve from a student with theoretical knowledge into a budding test engineer with practical, hands-on experience. I learned how to contribute to the quality and reliability of complex software systems, take ownership of testing tasks, and continually improve based on feedback. The experience has not only strengthened my technical skills but has also shaped my career aspirations by giving me a clear vision of what it means to be an effective and impactful SDET in the software industry.

One of the primary learning objectives of this internship was to gain hands-on experience in software testing, especially test automation in a real-world product environment. Prior to joining Engati Technologies, my knowledge of testing was mostly theoretical.

Through this internship, I learned how quality engineering fits into the broader software development lifecycle (SDLC) and how automation frameworks are practically implemented. I became familiar with testing strategies such as smoke testing, regression testing, and exploratory testing, and understood how they contribute to faster and more reliable software releases.

A significant focus was placed on improving my skills in automation testing using tools like **Nightwatch.js**, **Cucumber**, and **BrowserStack**. I learned how to write robust automated test cases following the **Page Object Model (POM)**, integrate those into CI/CD pipelines, and ensure their execution across multiple browsers and environments. This gave me a deeper understanding of how scalable and maintainable automation frameworks are built in modern QA practices. I also learned the importance of choosing the right test cases for automation and managing test data effectively.

Another key objective was to understand the process of defect tracking and test case management in agile projects. Using tools like **JIRA** and **TestRail**, I learned how to log detailed bug reports, assign priorities, and verify fixes through re-testing. Being part of daily stand-ups, sprint planning, and retrospectives helped me experience how quality assurance aligns with agile methodologies. I also gained communication skills through regular interactions with developers, product managers, and QA leads, which reinforced the importance of cross-functional collaboration.

QA Life Cycle (Agile Methodology) -

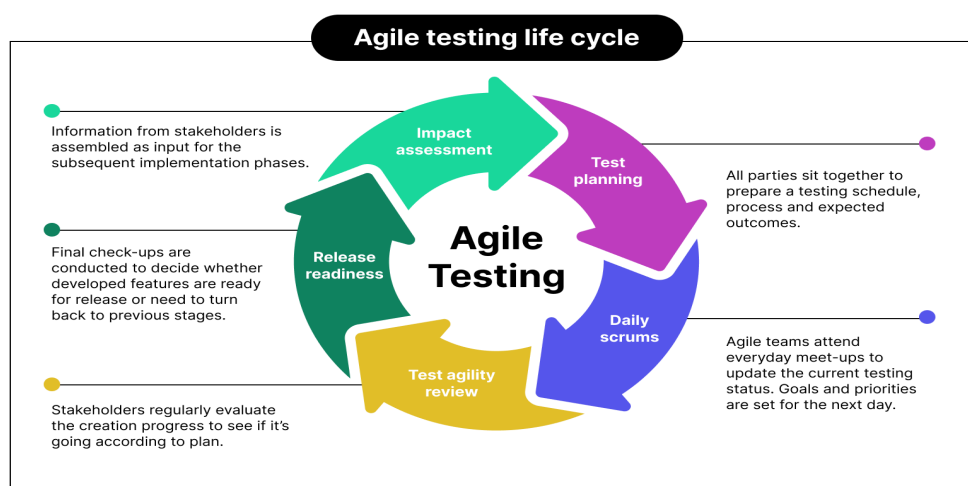


fig. 4.1 QA life cycle

The **QA Agile Methodology** is an approach to software quality assurance (QA) that is integrated into the agile development process. It emphasizes collaboration, flexibility, and continuous feedback, allowing testing and quality assurance activities to evolve alongside the development process. Here's a breakdown of key concepts and how QA functions within an agile environment:

1. Continuous Collaboration

In agile, QA is not a separate phase or activity that happens after development; it is an ongoing process that starts from day one of the project. Testers, developers, product owners, and business analysts work closely together, ensuring that quality is maintained throughout the development cycle. Testers are often involved in sprint planning meetings, backlog grooming, and daily stand-ups to discuss the progress of the project, clarify requirements, and identify potential issues early on. This collaboration ensures that all team members are on the same page and that any issues are caught quickly.

2. Iterative and Incremental Testing

Agile development is divided into **sprints**, typically lasting 1-2 weeks, during which small portions of the product are built, tested, and deployed. QA in agile follows this iterative approach by testing new features at the end of each sprint. Testers create test cases and run them on the new functionality, and if a defect is identified, it is immediately communicated to the development team for quick resolution. This process ensures that the application is continuously validated, preventing bugs from accumulating.

3. Test-Driven Development (TDD) and Behavior-Driven Development (BDD)

Two practices commonly used in agile QA are **Test-Driven Development (TDD)** and **Behavior-Driven Development (BDD)**. In TDD, developers write tests before writing the code itself, ensuring that the code meets the desired specifications. BDD, on the other hand, focuses on collaboration between developers, QA engineers, and non-technical stakeholders to define the expected behavior of a feature in simple, natural language. Tools like **Cucumber** and **Gherkin** are often used in BDD to create clear, understandable test scenarios that anyone can read and understand. Both practices ensure that quality is built into the product from the beginning.

4. Continuous Integration and Continuous Testing

In an agile environment, **Continuous Integration (CI)** and **Continuous Testing (CT)** are integral to the process. As developers frequently commit code, automated tests are run to ensure that the new code does not break existing functionality. This makes it easier to detect and fix bugs early in the development cycle. Automated testing plays a significant role in CI/CD pipelines, reducing the time it takes for developers to receive feedback and improving the overall quality of the software.

5. Early and Frequent Feedback

QA in agile emphasizes the importance of early and frequent feedback. Rather than waiting until the end of the development cycle to assess the product's quality, testing happens frequently throughout each sprint. Testers provide feedback on each iteration of the software, which allows the development team to make adjustments early on, reducing the risk of major issues down the line. This feedback loop fosters continuous improvement, which is a core principle of agile.

6. Flexibility and Adaptability

The agile methodology thrives on the ability to adapt to changing requirements. As features are developed and new ideas or challenges arise, the QA team must remain flexible. This may involve revising test cases, updating automation scripts, or even shifting testing priorities based on changes in the backlog. Agile QA requires a mindset that is open to change and quick to adjust to new directions in the project.

7. Quality as Everyone's Responsibility

In an agile team, quality is not just the responsibility of the QA team; it is a shared responsibility. Developers are responsible for writing clean, testable code, and product owners ensure that the requirements are clear and testable. The entire team works together to ensure the final product is of high quality, with QA engineers acting as advocates for best practices in testing.

CHAPTER 5

CHALLENGES AND SOLUTIONS

During my six-month internship as an SDET Intern at Engati Technologies Private Limited, I encountered a variety of technical and collaborative challenges that tested my problem-solving abilities, adaptability, and communication skills. Each challenge presented a unique learning opportunity and helped me grow both professionally and personally. Below are some of the key challenges I faced during the internship, along with the solutions and approaches I implemented to overcome them.

One of the initial challenges I encountered was **understanding the complexity of the Broadcast Module**, which was an integral part of Engati's AI chatbot platform. The module involved multiple workflows for scheduling, targeting, and dispatching bulk messages across various channels like WhatsApp, Messenger, and Web. As a newcomer, it took time to grasp how these flows operated, particularly under different use cases and conditions. To address this, I invested additional hours during the initial weeks to go through product documentation, participate in shadowing sessions with senior QA members, and ask clarifying questions in team meetings. This proactive learning approach helped me develop a strong conceptual understanding early on, which laid the foundation for effective testing later in the project.

Another major challenge was adapting to the **rapid development pace and agile workflows**. Engati followed a fast-moving agile environment with frequent sprint cycles and quick deployments. As a result, there were times when features had to be tested within tight timelines, leaving little room for exhaustive manual testing. To solve this, I focused on **prioritizing test cases** based on business-critical functionalities and impact areas. I also started building **automation scripts using Selenium and Cypress** for repetitive tasks to accelerate testing during regression cycles. Over time, this significantly improved testing speed and allowed the team to maintain quality even under time constraints.

I also faced difficulties when dealing with **intermittent and hard-to-reproduce bugs**, particularly during backend API testing. Some test scenarios behaved inconsistently due to race conditions, asynchronous message processing, or dependency on third-party services. These issues were not easy to replicate in a controlled test environment. To tackle this, I began logging detailed request-response data, introduced delays and retries in my automation scripts where necessary, and collaborated closely with backend developers to understand API behavior at a deeper level. .

Working on **performance testing** for the Broadcast Module introduced another layer of complexity. The system needed to handle thousands of messages in real time, which posed challenges in designing realistic load testing scenarios. Initially, I struggled to configure tools like **JMeter** to simulate concurrent message traffic without server throttling or timeout issues. I overcame this by researching best practices in performance testing, seeking guidance from senior QA engineers, and gradually scaling the test environment with controlled parameters. I learned to fine-tune request patterns and interpret performance metrics like response time, throughput, and system resource utilization. This hands-on experience enhanced my confidence in evaluating system performance under load.

Communication also posed a challenge during the early phase of the internship. As someone new to working in a professional setting, I initially hesitated to speak up during meetings or report issues assertively. However, I quickly realized that clear and timely communication was essential for efficient team collaboration. With guidance from my mentor, I started maintaining detailed documentation of my work, actively participating in daily stand-ups, and clearly articulating bugs or concerns through platforms like **JIRA** and **Slack**. This improved transparency, built trust with my teammates, and helped me grow as a more responsible and communicative team member.

Finally, balancing **multiple tools and technologies** was another challenge I faced. Switching between Selenium, Cypress, Postman, JMeter, and version control systems like Git required a strong learning curve. Initially, managing different environments and toolchains led to confusion and inefficiencies. To solve this, I created a personal knowledge base of commands, configurations, and testing workflows. I also developed small proof-of-concept projects to practice each tool individually before integrating them into the larger test suite. This organized approach helped me gain proficiency with each tool and work more efficiently over time.

One of the first challenges I encountered during my internship at Engati Technologies was adapting to the company's existing automation framework, which was built using Nightwatch.js and Cucumber. Coming from an academic background with limited exposure to JavaScript-based testing tools, I initially found it difficult to understand the structure of the test scripts and how different components interacted. To overcome this, I dedicated time outside of my daily tasks to study Nightwatch and Cucumber documentation, reviewed internal codebases, and practiced by writing small test cases. With guidance from my mentor and consistent hands-on practice, I became confident in contributing to the existing framework and even optimized a few redundant scripts.

Another major challenge was working with the Page Object Model (POM) for the first time. While the concept seemed straightforward, implementing it in a real-world project required careful planning and understanding of code reusability. Initially, my test cases were not modular, leading to repetitive code and difficulty in maintenance. After receiving constructive feedback during code reviews, I restructured my test scripts using POM best practices. This not only improved code readability and maintainability but also helped me appreciate the importance of design patterns in automation.

A recurring challenge I faced was dealing with flaky or inconsistent test results, especially when executing automated tests on BrowserStack. Tests would occasionally fail due to environment issues, such as slow internet, page load delays, or differences in browser behavior. To resolve this, I implemented explicit waits, handled dynamic elements more efficiently, and added better error logging to pinpoint the root causes of failures. Collaborating with the QA team, we also updated the CI pipeline to rerun failed tests and isolate environment-related issues from actual bugs, which significantly reduced false negatives.

Lastly, managing time and prioritizing tasks in a fast-paced agile environment was initially overwhelming. With regular sprint deadlines, code reviews, and test execution responsibilities, it was important to stay organized and efficient. I adopted tools like JIRA to break down my tasks into smaller subtasks and used personal checklists to track daily goals. Through this process, I learned how to estimate time accurately, communicate blockers early, and manage workload effectively. This experience not only improved my time management but also gave me valuable insight into how agile teams operate under tight release cycles.

One of the significant challenges I faced during my internship at Engati Technologies was **meeting tight deadlines** while maintaining the quality of my testing tasks. Working in an Agile environment meant that features were developed and released rapidly in short sprint cycles, often leaving limited time for thorough testing. At times, I had to balance writing new automation scripts, executing regression tests, and reporting bugs—all within a few days. This time pressure occasionally led to a trade-off between speed and depth, especially when last-minute changes were introduced close to the release date.

To address this, I developed better **time management and prioritization skills**. I started organizing my tasks using sprint boards and breaking them down into smaller subtasks with realistic time estimates.

I also communicated more actively with developers to gain early insights into features so I could prepare test cases in advance. Additionally, I utilized automation to handle repetitive testing, which freed up more time for exploratory and edge-case testing. This approach helped me meet deadlines more consistently without compromising the quality of my work. I encountered was managing the **unexpected scope changes** in the project.

Agile methodologies can sometimes lead to mid-sprint adjustments, where new features or bug fixes are added unexpectedly. These changes would often require updating or creating new test cases, which could push back the testing timelines. Balancing these sudden shifts with the existing testing workload created pressure, particularly when multiple tasks were dependent on each other to ensure comprehensive testing coverage. However, by staying adaptable, frequently communicating with the team, and continuously refining my approach to prioritization, I was able to stay on track and meet key milestones.

CHAPTER 6

ACHIEVEMENTS AND CONTRIBUTION

My internship as a Software Development Engineer in Test (SDET) at **Engati Technologies Private Limited** provided me with a platform to apply my skills in real-world projects and actively contribute to the organization's quality assurance efforts. Throughout my six-month internship, I worked closely with the QA and development teams on a mission-critical module—**Broadcast**—which is used to send bulk messages to customers across multiple platforms. I made several meaningful contributions that helped improve the reliability, performance, and scalability of the system while also achieving key personal milestones in terms of skill development and professional growth.

One of my primary contributions was the design and execution of detailed **manual test cases** covering all the functional aspects of the Broadcast module. This included creating, scheduling, sending, and logging of bulk messages. I ensured comprehensive test coverage for both positive and negative scenarios, helping the team identify bugs early in the development cycle. My attention to detail and proactive reporting enabled faster resolution of critical issues and ensured a more stable release process.

In addition to manual testing, I made significant progress in **test automation**. I successfully developed multiple **Selenium-based scripts** for end-to-end UI testing and used **Cypress** for faster front-end validations. I also worked on automating common test flows such as broadcast creation, message approval, and status validation, which reduced the manual testing workload and improved the regression testing turnaround time. My automation efforts contributed to the overall improvement in test efficiency and helped in catching issues earlier in the CI/CD pipeline.

Another important area where I added value was **API testing**. I created robust and reusable collections using **Postman** and incorporated **Cypress for backend API validations**, ensuring that the data flowing between the client-side UI and server-side components was accurate and consistent. These tests helped uncover inconsistencies in API responses and improved backend stability. I also documented API test cases and provided feedback to the backend team regarding improvements in error handling and performance.

I also took initiative in setting up and executing **performance tests** using **Apache JMeter** to simulate high-load conditions on the Broadcast system. I designed test scenarios that mimicked the sending of thousands of messages concurrently and analyzed the impact on server response time, CPU usage, and memory performance. This testing helped identify potential bottlenecks and informed the development team of optimizations required before production deployment.

Throughout the internship, I demonstrated consistent ownership by ensuring that all bugs were tracked, reproduced, and verified after fixes. I actively used **JIRA** for defect management and created clear and detailed bug reports that helped developers resolve issues efficiently. My collaboration with the development team was highly appreciated, particularly during tight sprint deadlines when clear communication and fast turnaround were critical.

I also contributed to **internal QA documentation**, including test case repositories, test plans, and automation guides. These resources served as valuable references for future testers and helped maintain transparency and continuity within the team.

On a personal level, I achieved several milestones, including becoming confident in using modern QA tools, understanding real-world agile workflows, and developing a problem-solving mindset. I improved my communication skills, learned to manage my time effectively, and became comfortable in a fast-paced, collaborative work environment.

I was able to make meaningful contributions to the Quality Engineering team while also achieving several personal and professional milestones. One of my key achievements was successfully developing and automating a suite of test cases for a critical chatbot workflow using Nightwatch.js and Cucumber. These tests were integrated into the CI/CD pipeline, enabling early bug detection and reducing manual testing effort by over 30% for that module. I also helped configure test execution on BrowserStack, allowing the team to ensure cross-browser compatibility more efficiently.

Another notable contribution was my involvement in improving the Page Object Model (POM) structure of the automation framework. I refactored several scripts to enhance modularity, reusability, and readability, which made future test case additions more streamlined. My updates were later used as a reference for junior team members onboarding the framework. Additionally, I documented my approach and shared it with the team during an internal knowledge-sharing session, which was well received.

I also contributed to manual testing efforts during sprint cycles by writing comprehensive test cases for new features and conducting regression testing before production releases. By identifying and reporting critical UI and functional bugs through JIRA, I played a role in maintaining the quality of high-visibility releases. My attention to detail and clear bug reports helped developers resolve issues faster, leading to fewer post-release defects.

Beyond technical tasks, I actively participated in daily stand-ups, sprint planning, and retrospectives. I regularly shared testing updates, raised blockers, and provided input on improving test coverage and timelines. These interactions helped build my confidence in professional communication and made me feel like a valued member of the team. Overall, my internship at Engati allowed me to contribute to impactful projects while growing as a software quality professional.

During my internship, I initiated a side project to set up and implement **Selenium Grid** for parallel test execution. This project aimed to reduce overall test execution time and simulate cross-browser testing by distributing tests across multiple environments. I chose this approach after noticing that running all automation tests sequentially led to longer feedback cycles, especially during regression phases. Setting up the Grid allowed me to understand the distributed architecture of Selenium, including the roles of the Hub and Nodes.

To begin, I configured the **Selenium Hub** on a central machine and connected multiple **Node instances** representing different browsers (like Chrome and Firefox) and platforms. I designed the test scripts in such a way that they could be executed concurrently without conflict, ensuring that tests remained stateless and independent. By using Selenium's capability to route tests dynamically, I was able to verify application behavior in parallel across different browsers, reducing execution time by more than 50%.

I integrated the parallel testing setup with a basic CI process to automatically trigger tests when code was pushed to a test branch. This gave me hands-on experience with real-world testing challenges such as session management, port conflicts, synchronization issues, and logging in a distributed environment. To ensure scalability, I documented the setup so other team members could replicate it or expand it to include additional browsers or cloud-based nodes like BrowserStack or Sauce Labs.

This project not only improved my understanding of **test automation infrastructure** but also demonstrated how advanced tools and configurations can optimize QA workflows. It helped me bridge the gap between test script writing and full-scale test execution strategies, adding depth to my internship experience. The successful implementation of this project was acknowledged by my mentor and opened up discussions on integrating similar concepts into Engati's existing automation pipeline. I designed the test scripts in such a way that they could be executed concurrently without conflict, ensuring that tests remained stateless and independent. By using Selenium's capability to route tests dynamically, I was able to verify application behavior in parallel across different browsers, reducing execution time by more than 50%.

CHAPTER 7

REFLECTION AND ANALYSIS

Looking back on my six-month internship as a Software Development Engineer in Test (SDET) at **Engati Technologies Private Limited**, I can confidently say that the experience was transformative in both a technical and personal sense. It provided me with a unique opportunity to transition from a student mindset to a professional approach, and helped me better understand the expectations, responsibilities, and challenges of working in the software industry.

From a technical perspective, I entered the internship with basic theoretical knowledge of software testing concepts, but emerged with practical, hands-on experience in a real product environment. Being part of the team that tested and supported the **Broadcast Module** of Engati's AI chatbot platform was both challenging and rewarding. I learned how critical it is for software to be robust, scalable, and user-friendly—especially when it is used to communicate with large numbers of customers in real time.

Throughout the internship, I gained confidence in using industry-standard tools like **Selenium, Cypress, Postman, and JMeter**. More than just learning how to use these tools, I came to understand **why** they are used, **where** they add value, and **how** to integrate them effectively into a test workflow. This practical knowledge is something I could never have acquired through textbooks alone. I also became more comfortable with source control systems like **Git**, ticketing tools like **JIRA**, and working within **agile sprint cycles**, which helped me understand how software products evolve iteratively through collaboration between teams.

One of the most significant realizations during the internship was the importance of **communication and teamwork**. Initially, I was hesitant to speak up in meetings or ask questions, fearing that I might seem inexperienced. However, over time, I learned that asking the right questions and expressing concerns early often leads to better outcomes and faster problem resolution. I also appreciated how important it is to **document work clearly**, as this ensures smooth handovers and helps maintain the continuity of testing efforts, even in dynamic team environments. Reflecting on my internship journey at Engati Technologies, I realize how transformative the experience has been for my personal and professional growth. Coming into the role, I had theoretical knowledge of testing and software development, but this internship gave me the platform to apply that knowledge in a real-world, fast-paced tech environment.

I not only gained hands-on experience in test automation using industry-standard tools like Nightwatch.js and Cucumber but also learned how software quality assurance is deeply integrated into the agile development process.

Working on live projects taught me the importance of writing clean, maintainable, and scalable code. It also made me appreciate the value of proactive communication, collaboration, and documentation in ensuring a smooth development cycle. The challenges I faced—whether it was debugging flaky tests, adapting to the Page Object Model, or working with cross-browser testing on BrowserStack—pushed me to go beyond just solving problems and start thinking critically about long-term solutions and best practices.

One of the most valuable aspects of this internship was the opportunity to work alongside experienced QA engineers and developers. Their feedback during code reviews, their approach to solving problems, and their clarity during meetings shaped how I now approach tasks, break down problems, and prioritize quality. I learned that being a good SDET is not just about writing automated scripts—it's about understanding user behavior, anticipating edge cases, and contributing to a culture of continuous improvement.

Overall, this internship has strengthened my confidence, improved my technical and soft skills, and confirmed my passion for pursuing a career in software quality engineering. It gave me a real sense of responsibility and the opportunity to make meaningful contributions to a product used by real customers. I leave this internship with a deeper understanding of how technology, testing, and teamwork come together to build reliable and user-friendly software.

Throughout my internship, I developed a greater understanding of the challenges and complexities that come with testing AI-driven systems. Testing a conversational AI platform, particularly a chatbot, involves unique considerations that go beyond traditional UI or functional testing. I had to account for multiple variables, such as user inputs, natural language processing (NLP), and dynamic conversation flows.

This required a different approach to testing, where traditional UI-based test cases were adapted to ensure the chatbot's responses, intents, and user flows were functioning as expected. The ability to think critically about the different ways users might interact with the bot and how the system would respond helped me appreciate the nuances of testing AI and machine learning systems, which I had not previously encountered in my academic training.

One significant realization I had during my internship was the importance of adaptability in the face of rapidly changing requirements. As the development team iterated on features, testing often had to be adjusted to account for last-minute changes or bug fixes. I quickly learned that flexibility, along with a proactive mindset, was crucial in ensuring that the test cases remained relevant and effective. Moreover, working in an agile environment meant that I had to balance testing activities with tight deadlines, necessitating prioritization and efficient time management. Reflecting on this, I now understand how valuable it is to be adaptable, organized, and able to quickly learn and pivot when unexpected changes arise, which are common in fast-paced tech environments.

CHAPTER 8

RECOMMENDATIONS

1. Expand Test Automation Coverage

While automation is already in place for several workflows, expanding automated test coverage to include more complex user journeys, edge cases, and negative scenarios will further reduce the need for repetitive manual testing. A greater focus on API automation and integration-level tests would also help catch defects early in the development lifecycle.

2. Implement Centralized Test Case Management

Introducing a centralized test case management tool such as TestRail, Zephyr, or Xray (integrated with JIRA) would help organize, track, and update test cases more efficiently. This would improve test traceability and simplify collaboration between QA, development, and product teams.

3. Enhance Performance Testing Strategy

The Broadcast module handles a high volume of outbound messages and should be stress-tested regularly. Setting up automated performance test pipelines using tools like JMeter, Gatling, or Locust integrated with CI/CD would ensure consistent monitoring of system scalability and resource utilization under load.

4. Foster Cross-Functional Training

Encouraging cross-training between QA, development, and product teams can improve understanding of system architecture, reduce testing blind spots, and enable better prioritization of test efforts. This also helps foster a more collaborative and flexible work culture.

5. Strengthen Onboarding Resources

Enhancing onboarding documentation for interns and new QA members—such as setup guides, environment configurations, and common testing practices—can reduce ramp-up time and enable quicker contribution to projects.

Based on my experience during the SDET internship at Engati Technologies, one key recommendation is to continue investing in the enhancement of the existing automation testing framework. While the current setup using Nightwatch.js and Cucumber is functional and effective, there is scope for improving test execution time and reliability. Incorporating parallel test execution more effectively and optimizing selectors can further reduce flakiness. Additionally, introducing unit tests for custom utilities and frequently used components within the automation suite can help prevent small changes from breaking core functionalities. Over time, transitioning to a more scalable framework such as Cypress or Playwright could also be explored, as these offer improved debugging, faster execution, and modern JavaScript support.

Another recommendation is to implement a structured onboarding guide for new interns and team members in the QA department. When I first joined, I spent a considerable amount of time understanding the folder structure, test execution flow, and build setup. Having a detailed onboarding document or walkthrough video would significantly reduce the learning curve and improve productivity from the start. This could include guidelines on setting up the test environment, naming conventions, common troubleshooting steps, and a glossary of terms and tools used across the QA and DevOps teams. Introducing this documentation as a shared resource would also ensure knowledge continuity when team members transition.

From a process standpoint, I recommend improving the communication flow between developers and testers during sprint cycles. While agile practices like daily stand-ups are already followed, dedicated QA-dev sync-up sessions during sprint mid-points can help clarify edge cases, reduce misinterpretation of requirements, and speed up bug resolution. Sometimes, test cases needed to be adjusted after development due to last-minute changes in the implementation. By encouraging joint review sessions for user stories or pairing a tester with a developer for story validation, teams can improve collaboration and reduce rework. This also aligns with shift-left testing principles, ensuring that quality is built in from the early stages.

Lastly, it would be beneficial for Engati Technologies to expand the scope of QA to include more non-functional testing, such as performance, accessibility, and security testing. While functional testing ensures correctness, non-functional areas are increasingly critical for user experience and compliance. Integrating tools like Lighthouse for accessibility and performance audits or running periodic OWASP checks for security would help the platform stay competitive and user-friendly. Providing QA interns with exposure to these areas will not only strengthen the product but also broaden their skillsets. In the long term, such enhancements would contribute to a more mature, resilient, and comprehensive testing strategy across the organization.

CHAPTER 9

CONCLUSION

My internship at **Engati Technologies Private Limited** has been an incredibly enriching and transformative experience. Over the course of six months, I had the opportunity to immerse myself in the professional world of software testing, work alongside skilled engineers, and contribute meaningfully to the development and quality assurance of a real-world AI chatbot platform.

Working specifically on the **Broadcast Module**, a critical feature that enables businesses to send bulk messages to their customers, allowed me to apply a wide range of testing techniques—manual, automation, API, and performance testing—in a dynamic and fast-paced environment. I gained practical experience with tools like Selenium, Cypress, Postman, and JMeter, and developed a deeper understanding of agile methodologies, defect lifecycle management, and continuous testing practices.

More importantly, this internship taught me the importance of communication, collaboration, and attention to detail. I learned to take ownership of tasks, proactively identify issues, and present solutions that added real value to the project. Through the support of my mentors and teammates, I built the confidence to express ideas, ask questions, and continuously improve both my technical and interpersonal skills.

This experience has not only strengthened my foundational knowledge but has also shaped my long-term career interests in quality engineering and test automation. I now feel better prepared to take on future challenges in the software industry and contribute meaningfully to high-quality product delivery.

In conclusion, my time at Engati was much more than an internship—it was a vital stepping stone toward becoming a responsible, capable, and quality-focused software professional. I am truly grateful for the exposure, mentorship, and learning opportunities provided to me, and I look forward to applying these lessons as I continue to grow in my career.

Through this internship, I enhanced my technical skills in writing and executing automated test cases using Nightwatch.js and Cucumber, managing test scripts through the Page Object Model, and utilizing tools such as BrowserStack, Git, and JIRA. I also developed a structured approach to problem-solving, debugging test failures, and prioritizing tasks effectively within an agile workflow.

These skills have not only strengthened my professional capabilities but also increased my confidence in taking on more complex responsibilities in future roles.

More importantly, this internship taught me the value of teamwork, communication, and continuous learning. Collaborating with developers, QA engineers, and mentors helped me understand the dynamics of cross-functional teams and the critical role each member plays in delivering a reliable product. The feedback and guidance I received throughout the internship contributed immensely to my growth as a quality-focused engineer.

In conclusion, my time at Engati Technologies has laid a solid foundation for my career in software quality engineering. It has equipped me with practical skills, real-world experience, and the confidence to pursue future opportunities in the field of software testing and development. I am grateful for the exposure and mentorship I received and look forward to applying the lessons learned in my continued professional journey.

CHAPTER 10

REFERENCES

- **Nightwatch.js Documentation**

Nightwatch.js – Automated testing framework for web applications and websites

<https://nightwatchjs.org>

- **Cucumber.js Documentation**

Behavior-driven development (BDD) with Cucumber for JavaScript

<https://cucumber.io/docs/guides/10-minute-tutorial/>

- **JIRA Software by Atlassian**

Agile project management tool used for bug tracking and task planning

<https://www.atlassian.com/software/jira>

- **Page Object Model Design Pattern**

Best practices for maintaining test automation code

https://www.selenium.dev/documentation/test_practices/encouraged/page_object_model

- **Engati Technologies Website**

Official website of Engati, provider of conversational AI and chatbot platforms

<https://www.engati.com>

- **OWASP Foundation**

Security testing principles and top 10 vulnerabilities

<https://owasp.org>